collect_daily_close.py — 코드만 (8주차)

설명(주석)을 걷어내고 코드 줄만 남긴 것입니다. 구조를 한눈에 보라고 만든 자료이고,
같은 폴더의 collect_daily_close.py 이 원본입니다 — 주석은 그 파일에 그대로 있습니다.
한 줄이 길어 넘어간 곳은 다음 줄에 안쪽으로 들여 이어 붙였습니다.
이모지(📨 📨 🌐 🌐 …)는 이 PDF 글꼴에 없어 네모로 보입니다 — 원본에는 정상입니다.
※ 굵은 글씨 = 이번 주에 새로 생긴 줄입니다 (49줄).
   bigd_07_chart 폴더의 파일들에 없던 줄만 굵게 했습니다 — 지난주 코드를 다른 파일로
   옮겨 온 줄은 새 코드가 아니므로 보통 글씨입니다.
==

```
import os
import time
import requests
from datetime import datetime
from zoneinfo import ZoneInfo
from dotenv import load_dotenv

WATCHLIST_FILE = "watchlist.csv"
NUM_HISTORY_DAYS = 15
KST = ZoneInfo("Asia/Seoul")

load_dotenv()

TURSO_DATABASE_URL = os.environ.get("TURSO_DATABASE_URL")
TURSO_AUTH_TOKEN = os.environ.get("TURSO_AUTH_TOKEN")


def read_watchlist(path: str = WATCHLIST_FILE) -> list:
    if not os.path.exists(path):
        print(f"📨 '{path}' 파일이 존재하지 않습니다.")
        return None

    try:
        import pandas as pd

        watchlist_df = pd.read_csv(path, dtype={"code": str})

        watchlist_df.columns = [col.strip().lower() for col in watchlist_df.columns]

        if "code" not in watchlist_df.columns:
            print("📨 CSV 파일에 'code' 컬럼이 존재하지 않습니다.")
            return None

        codes = watchlist_df["code"].tolist()
        print(f"🌐 읽어온 관심 종목 리스트: {codes}")
        return codes

    except Exception as e:
        print(f"📨 CSV 파일을 읽는 동안 오류가 발생했습니다: {e}")
        return None


def fetch_naver_current_price(code: str, retries: int = 2) -> dict:
    url = f"https://m.stock.naver.com/api/stock/{code}/basic"
    for attempt in range(1, retries + 1):
        try:
            response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"},
```

```
                    timeout=3)
            if response.status_code != 200:
                print(f"⚠ [{code}] 네이버 현재가 조회 실패 (응답 코드: {response.status_code},
                        {attempt}/{retries}번째 시도)")
            else:
                data = response.json()
                price_str = data.get("closePrice", "0").replace(",", "")
                price = int(price_str) if price_str.isdigit() else 0
                if price <= 0:
                    print(f"⚠ [{code}] 가격을 숫자로 읽지 못했습니다 "
                          f"(closePrice: {data.get('closePrice')!r},
                                  {attempt}/{retries}번째 시도)")
                else:
                    return {
                        "code": code,
                        "name": data.get("stockName", "알 수 없음"),
                        "price": price,
                        "rate": float(data.get("fluctuationsRatio", "0") or "0"),
                        "is_open": data.get("marketStatus") == "OPEN",
                        "traded_at": data.get("localTradedAt"),
                    }
        except Exception as e:
            print(f"⚠ [{code}] 네이버 현재가 조회 중 오류 발생: {e} ({attempt}/{retries}번째 시도)")

        if attempt < retries:
            time.sleep(1)

    return None


def get_turso_client():
    import libsql_client

    if not TURSO_DATABASE_URL:
        raise RuntimeError("TURSO_DATABASE_URL 환경변수가 설정되지 않았습니다.")

    url = TURSO_DATABASE_URL.replace("libsql://", "https://", 1)
    client = libsql_client.create_client_sync(url, auth_token=TURSO_AUTH_TOKEN)
    client.execute(
        "CREATE TABLE IF NOT EXISTS price_history ("
        "code TEXT NOT NULL, date TEXT NOT NULL, close INTEGER NOT NULL, "
        "PRIMARY KEY (code, date))"
    )
    return client


def update_price_history(code: str, date_str: str, close_price: int,
                         num_days: int = NUM_HISTORY_DAYS, client=None) -> None:
    own_client = client is None
    if own_client:
        client = get_turso_client()
    try:
        client.execute(
            "INSERT INTO price_history (code, date, close) VALUES (?, ?, ?) "
            "ON CONFLICT (code, date) DO UPDATE SET close = excluded.close",
            [code, date_str, close_price],
        )
        client.execute(
```

```
            "DELETE FROM price_history WHERE code = ? AND date NOT IN ("
            "SELECT date FROM price_history WHERE code = ? ORDER BY date DESC LIMIT ?)",
            [code, code, num_days],
        )
    except Exception as e:
        print(f"❌ [{code}] Turso DB에 종가 저장 실패: {e}")
    finally:
        if own_client:
            client.close()


watchlist_codes = read_watchlist()
if watchlist_codes is None:
    exit()

print(f"📡 종가 수집 시작... (실행 시각 {datetime.now(KST).strftime('%Y-%m-%d %H:%M')} KST)")

collected = 0
with get_turso_client() as client:
    for code in watchlist_codes:
        info = fetch_naver_current_price(code)
        if info is None:
            continue

        if info["is_open"]:
            print(f"⚠[{code}]  아직 장중입니다. 확정되지 않은 가격이라 기록하지 않습니다.")
            continue

        traded_at = info.get("traded_at") or ""
        if len(traded_at) >= 10:
            date_str = traded_at[:10].replace("-", "")
        else:
            date_str = datetime.now(KST).strftime("%Y%m%d")
            print(f"⚠[{code}]  체결 시각을 읽지 못해 실행 시각 기준으로 날짜를 정합니다.")

        update_price_history(code, date_str, info["price"], client=client)
        collected += 1
        print(f"✅ [{code}] {info['name']} {date_str} 종가 {info['price']:,}원 기록")

if collected == 0:
    print("❌ 기록된 종가가 하나도 없습니다.")
    exit()

print(f"🌐 종가 히스토리를 Turso DB에 저장했습니다 ({collected}/{len(watchlist_codes)}종목).")
```